Bilgisayar Programlama

Bellek Yönetimi ve Pointer Kavramı

İçindekiler

1. Bellek Mimarisine Giriş
   - Değişkenler RAM'de Nerede Yaşar? (Adres ve Byte)
   - Ampersand (`&`) Operatörü
2. Pointer (İşaretçi) Kavramı ve Tanımlanması
   - Pointer Nedir? (`*` Asterisk Operatörü)
   - İzleme (Trace) Tablosu
3. Değer vs. Referans ile Çağırma
   - Call by Value ve Call by Reference
   - İzleme Tablosu, Stack Bellek Görünümü
   - Klasik `swap()` Fonksiyonu
   - *Mekatronik Pratik:* Çoklu Değer Döndürme

İçindekiler (Devam)

4. Diziler ve Pointer İlişkisi (Array Decay)
   - İşaretçi Aritmetiği (RAM'de Adım Atmak)
   - Fonksiyona Dizi Aktarmak
   - Pointer Boyutu (`sizeof`)
   - `const` Pointer — Salt Okunur Erişim
5. Sık Karşılaşılan Hatalar ve Çözümleri
   - Derleme Hatası Kataloğu, Mantık Hatası Kataloğu
6. Uygulamalı Laboratuvar Çalışmaları

BÖLÜM 1

Bellek Mimarisine Giriş

1.1 Değişkenler RAM'de Nerede Yaşar?

Kod yazdığımızda değişkenler sihirli bir kutuda değil, doğrudan mikrodenetleyicinin RAM (Rastgele Erişimli Bellek) hücresinde tutulur. Her bir hücre 1 Bytelık (8 bit) veri saklar. Ve en önemlisi: Her hücrenin eşsiz bir adresi (Kapı Numarası) vardır.

```
Bellek Adresleri (Genellikle Hexadecimal / 16'lık tabanla gösterilir)
                +-----------+
 0x1000         |  b1       | <- 1 Byte (Örn: char kullanıldı)
                +-----------+
 0x1001         |  b2       |
                +-----------+
 0x1002         |           | \
 0x1003         |  156      | |-> 4 Byte ardışık alan kullanıldı!
 0x1004         |  (int x)  | |   (Örn: int x = 156;)
 0x1005         |           | /
                +-----------+
```

Adres neden önemli? Postacı mantığı... 156 değerine erişmek istiyorsanız, bellek adresinin `0x1002` numarasından başladığını bilmek C dilinin donanımı yönetmedeki gücüdür.

1.2 "Nerede" Olduğunu Bulmak: Ampersand (`&`) Operatörü

C dilinde bir değişkenin değerini (içeriğini) okumayı biliyoruz. Ancak değişkenin RAM'deki **Adresini (Nerede olduğunu)** öğrenmek için önüne `&` operatörü koyarız.

```
int motorPWM = 180;

printf("Degeri nedir (Veri)  : %d\n", motorPWM);        // Ekrana 180 yazar
printf("Nerede yasiyor (Adres): %p\n", &motorPWM);      // Ekrana örn: 0x7ffeefbff5ac ya
```

Hatırlatma: `scanf("%d", &motorPWM)` kullanırken neden `&` koyuyorduk? Çünkü klavyeden okunan değeri fonksiyona DEĞER (copy/180) olarak değil, doğrudan RAM'de DEĞİŞTİRMESİ için ADRESİ (kapı numarasını) kuryeyle gönderiyorduk!

BÖLÜM 2

Pointer (İşaretçi) Kavramı

2.1 Pointer Nedir? (`*` Operatörü)

Pointer, içinde 10, 20.5 veya 'A' tutan standart bir değişken değildir. Pointer, içinde ADRES tutan (*Başka bir değişkenin adresini saklayan*) özel bir ajandır.

Bir pointer tanımlamak ya da işaret ettiği "Adres'in İçindeki Veriye" ulaşmak (oku/değiştir) için başına `*` (Asterisk) operatörü konulur.

```c
int x = 25;       // Standart degisken. Icinde 25 var. RAM adresi: 0x1000

// POINTER TANIMLAMA: "Ben int bir degiskenin adresini tutacağım!"
int *ptr_isaretci;

// POINTER'A ADRES YUKLEME
ptr_isaretci = &x;  // ptr_isaretci'nin icinde artik 0x1000 degeri var.

// POINTER ILE ORIJINAL VERIYE UZAKTAN ERISIM (Dereference)
*ptr_isaretci = 99; // 0x1000 adresine git, icindeki veriyi PARÇALA ve 99 yap.

printf("x'in yeni degeri: %d", x); // x artik 99 oldu!
```

2.2 Pointer İzleme (Trace) Tablosu

Yukarıdaki işlemi RAM gözünden canlandıralım:

Kod Adımı	`x` 'in içindeki (Value)	`&x` (Adres)	`ptr_isaretci` 'nin tuttuğu bilgi
`int x = 25;`	25	0x1000	`[Çöp Değer - Garbage]`
`int *ptr = &x;`	25	0x1000	`0x1000` (x'in adresini öğrendi)
`*ptr = 99;`	99	0x1000	`0x1000` (Adrese uzaktan suikast yaptı!)

Mekatroniğe Uyarlama Konsepti: Eğer `0x1000` kod adresinin bulunduğu bölge, bir robotun **MOTOR-1 KONTROL ALANI** olan bir donanım bölgesi (Register / Yazmaç) olsaydı? Araya fonksiyonlar girmesine gerek kalmadan, hafıza adresini bilmemiz koşuluyla doğrudan `*ptr = 255;` diyerek donanımın tam güce çıkmasını / kilidinin açılmasını sağlardık (STM32, Arduino Direct Port Access felsefesi).

BÖLÜM 3

Değer (Value) vs. Referans (Reference) ile Çağırma

3.1 Fonksiyonlarda "Call by Value" Kayıtları

Bir fonksiyona klasik parametre gönderdiğinizde, aslında değişkenleri değil, sadece **fotokopilerini** gönderirsiniz. İşlem fotokopi üzerinde yapılır, orijinal değişken değişmez.

× Problem (Değiştirilemeyen Sensör Verisi):

```
void sensorKonumuGuncelle(int okunan) {
    okunan = okunan + 10; // Fotokopinin uzerine +10 yazildi.
}

int main() {
    int x_ekseni = 5;
    sensorKonumuGuncelle(x_ekseni); // Fotokopi gönderildi (Değer)

    printf("Eksen: %d\n", x_ekseni); // Hala 5'tir! Ana değer değişmedi.
    return 0;
}
```

Eğer parametre sayısını ve dönecek `return` miktarını birden fazlaya genişletmek istersek bu yöntem yetersiz kalır.

3.2 Orijinali Değiştirmek: "Call by Reference" (Adres Aktarımı)

Fonksiyonun bellekteki ASIL veriyi (fotokopiyi değil) etkilemesi için fonksiyona verinin `&` kullanarak adresini göndermeliyiz. Fonksiyon tarafı da karşılama yaparken Pointer `*` parametresi bırakmalıdır.

✅ Çözüm (Pointer ile güce erişim):

```c
// Fonksiyon Diyor Ki: "Bana deger degil, int adresi yolla (*). Ben onu iceriden vururum
void sensorKonumuGuncelle(int *ptr_okunan) {
    // Uzaktan adrese git, eski verideki değeri Al, 10 ekle ve Orjinal yere Yapıştır.
    *ptr_okunan = *ptr_okunan + 10;
}

int main() {
    int x_ekseni = 5;
    // Orijinal yeri, bellek adresini kargola (&)
    sensorKonumuGuncelle(&x_ekseni);

    printf("Eksen: %d\n", x_ekseni); // Ekranda 15! Başarıyla orijinal veriye sızıldı.
    return 0;
}
```

3.2.1 Call by Reference — İzleme Tablosu

`sensorKonumuGuncelle(&x_ekseni)` çağrısının adım adım izlenmesi:

Adım	Konum	İşlem	`x_ekseni` (main)	`ptr_okunan` (fonksiyon)
1	`main`	`x_ekseni = 5` tanımlandı	5	—
2	`main → fonksiyon`	`&x_ekseni` (adres) kopyalandı	5	0x1000
3	`fonksiyon`	`*ptr_okunan = *ptr_okunan + 10`	15	0x1000
4	`fonksiyon → main`	Fonksiyon sonlandı, `ptr_okunan` silindi	15	*(silindi)*

Fark: Call by Value'da orijinal `x_ekseni` hep 5 kalıyordu. Call by Reference'da adres bilgisi sayesinde orijinal doğrudan değiştirildi.

3.2.2 Stack Bellek Görünümü — Call by Reference

```
Stack (Çağrı Anında):
┌──────── main() ──────────────────┐   ┌── sensorKonumuGuncelle() ──────┐
│ Adres: 0x1000                    │   │ Adres: 0x2000                  │
│                                  │   │                                │
│  ┌──────────────┐                │   │  ┌──────────────┐              │
│  │ x_ekseni = 5 │ ◄──────────────┼───┼  │ ptr_okunan =  │              │
│  └──────────────┘                │   │  │   0x1000     │              │
│                                  │   │  └──────────────┘              │
│ x_ekseni DEĞİŞTİ → 15            │   │ *ptr_okunan ile adrese yazıldı │
└──────────────────────────────────┘   └────────────────────────────────┘
```

Temel Fark: Call by Value'da **fotokopi (copy)**, Call by Reference'da **adres (GPS koordinatı)** gönderilir.

3.2.3 Klasik `swap()` Fonksiyonu — İki Değişkenin Takası

Sınavlarda en sık sorulan pointer sorusudur. İki değişkenin değerlerini **pointer ile** takas eder:

```c
void swap(int *a, int *b) {
    int gecici = *a;   // a'nın gösterdiği değeri yedekle
    *a = *b;           // b'nin değerini a'ya yaz
    *b = gecici;       // yedeklenen değeri b'ye yaz
}

int main() {
    int motor1_hiz = 100, motor2_hiz = 250;
    printf("Oncesi: M1=%d, M2=%d\n", motor1_hiz, motor2_hiz);

    swap(&motor1_hiz, &motor2_hiz);  // Adreslerini gönder

    printf("Sonrasi: M1=%d, M2=%d\n", motor1_hiz, motor2_hiz);
    // Çıktı: M1=250, M2=100 — Başarıyla takas edildi!
    return 0;
}
```

⚠️ Neden `swap(motor1_hiz, motor2_hiz)` çalışmaz? Çünkü fotokopi (Call by Value) gönderilir; orijinaller değişmez.

3.3 Mekatronik Uygulaması: "Çoklu Değer Döndürme İhtiyacı"

`return` sadece tek değer döndürebilir dahi demiştik. Ya bizim Fonksiyonumuz AYNI ANDA Lidar'ın Mesafe, Açı ve Güvenlik Statüsünü 3 farklı bilgiyi değiştirmesi gerekirse?

✅ Pointer (Adres Transferi) İle Çoklu Geri Dönüş Yapmak:

```c
void lidarDataGuncelle(float *ptr_mesafe, float *ptr_aci, int *ptr_guvenli_mi) {
    // Donanimdan UART ile verileri okudugumuzu varsayiyoruz...
    // Oku, dogrudan RAM adresine yapistir!
    *ptr_mesafe = 125.5;
    *ptr_aci = 45.0;
    *ptr_guvenli_mi = 1; // 1 = Safe (Guvenli)
}

int main() {
    float uzaklik = 0, yon_aci = 0;
    int durum = 0;

    // Asla Return kullanmadık! C dilinde "Void" bile olsa belleklerine sızdık.
    lidarDataGuncelle(&uzaklik, &yon_aci, &durum);

    printf("Mesafe: %f, Aci: %f, Durum: %d\n", uzaklik, yon_aci, durum);
    return 0;
}
```

BÖLÜM 4

Diziler ve Pointer İlişkisi (Array Decay)

4.1 Diziler Neden Hep "Pointer"dır? (Büyük Sır)

İçinde 1000 adet veri olan bir DİZİMİZ (`int ivmeolcer[1000]`) olduğunu düşünelim.

```
int dizi[5] = {10, 20, 30, 40, 50};
```

C Dilinin en ilginç kurallarından biri; "Dizinin ismi, aslında dizinin 0. (sıfırıncı) elemanının başlangıç hafıza adresini tutan sabit bir POINTER'dır."

```
// "dizi" demek, gizlice "&dizi[0]" demektir! İspat edelim:
printf("Dizinin Simgesi (Ismi): %p\n", dizi);
printf("0. Elemanın  Adresi   : %p\n", &dizi[0]);

// ÇIKTILAR BİREBİR AYNIDIR!
```

4.2 İşaretçi Aritmetiği (RAM'de Adım Atmak)

İşaretçileri tam sayılarla (örn: `ptr + 1`) toplayabiliriz. Ancak bu matematiksel bir toplama değil, **bellekte bir sonraki hücreye atlama** işlemidir. Atlama miktarı, veri tipinin boyutuna (`sizeof`) bağlıdır.

```c
int dizi[3] = {10, 20, 30};
int *ptr = dizi; // 0. elemanın adresi

printf("0. Eleman: %d (Adres: %p)\n", *ptr, ptr);
ptr++; // Bir sonraki int hücresine (4 byte ileri) git
printf("1. Eleman: %d (Adres: %p)\n", *ptr, ptr);

// *(ptr + i) aslında dizi[i] ile aynı şeydir!
printf("2. Eleman: %d\n", *(dizi + 2));
```

Mekatronik Notu: Sensörlerden gelen ardışık verileri (Dizi) bellek üzerinde yüksek hızla taramak için kullanılır.

4.3 Fonksiyona Dizi Aktarmak: Zaman ve RAM Optimizasyonu

Bir fonksiyona `func(dizi)` şeklinde yazdığınızda derleyici koca 1000 elemanı tek tek yorulup fotokopisini çekmez. **Asla fotokopi yapmaz, sadece dizinin "0. elemanının adresini" fonksiyona iletir.** (Bu sürece Array Decay to Pointer - Dizinin pointera bozunması denir).

Niçin muazzamdır? Eger 100 Sensör verisi fotokopi ile Kopyalansaydı CPU/RAM perişan olurdu! Mikrodenetleyicide gecikme olmadan (0 overhead) hız sağlar.

```
// Diziyi adres (pointer) seklinde yolladigimiz icin * isaretiyle karsilayabiliriz!
void sensorFiltresi(int *ptr_liste, int boyut) {
    // Orjinal Array'e doğrudan yazabiliyoruz demektir!
    for (int i = 0; i < boyut; i++) {
        ptr_liste[i] = ptr_liste[i] / 2; // Tüm dizi verilerini yarıya indir / gürültü a
    }
}

int main() {
    int adc[3] = {1000, 800, 600};
    // adc ismi bir ADRES/POINTER'dır, başına '&' koymamıza BİLE GEREK YOK!
    sensorFiltresi(adc, 3);
    return 0;
}
```

4.4 Pointer Boyutu — `sizeof` ile Doğrulama

Pointer'lar hangi veri tipini gösterirlerse göstersinler, tuttukları şey hep bir **bellek adresidir.** Bu nedenle tüm pointer'ların boyutu aynıdır:

```
printf("sizeof(int)    = %lu byte\n", sizeof(int));     // 4
printf("sizeof(float)  = %lu byte\n", sizeof(float));   // 4
printf("sizeof(double) = %lu byte\n", sizeof(double));  // 8

printf("sizeof(int*)   = %lu byte\n", sizeof(int*));    // 8 (64-bit)
printf("sizeof(float*) = %lu byte\n", sizeof(float*));  // 8 (64-bit)
printf("sizeof(char*)  = %lu byte\n", sizeof(char*));   // 8 (64-bit)
// Tüm pointer boyutları AYNI!
```

Platform	Adres Boyutu	Pointer Boyutu
32-bit MCU (STM32, Arduino)	32 bit	4 byte
64-bit PC (x86_64)	64 bit	8 byte

Sonuç: Pointer'ın boyutu işletim sisteminin adres uzayına bağlıdır, gösterdiği verinin tipine değil.

4.5 `const` Pointer — Salt Okunur Erişim

Bir fonksiyona pointer ile veri gönderdiğimizde, fonksiyon o veriyi hem okuyabilir hem de değiştirebilir. Peki ya fonksiyonun veriyi sadece okumasını ve değiştirmemesini istiyorsak?

```c
// const int *ptr: "Bu pointer üzerinden veri DEĞİŞTİRİLEMEZ!"
void sensorVeriYazdir(const int *ptr_veri, int boyut) {
    for (int i = 0; i < boyut; i++) {
        printf("Sensor[%d] = %d\n", i, ptr_veri[i]);
        // ptr_veri[i] = 0;  // ❌ Derleme Hatası! const ile korunuyor.
    }
}

int main() {
    int adc_ham[4] = {512, 768, 256, 1023};
    sensorVeriYazdir(adc_ham, 4);  // Veriyi güvenle gönder
    return 0;
}
```

Mekatronik Notu: Sensör kalibrasyonu gibi kritik verilerin "yanlışlıkla bozulmasını" engellemek için `const` pointer kullanılır. Gömülü yazılımda çok yaygındır.

BÖLÜM 5

Pointerların Güvenlik Sorunları (Gömülü Sistem Çökmeleri)

5.1 Wild (Başlatılmamış) ve Null Pointerlar

Pointerlar çok güçlüdür (Spiderman Kuralı: Büyük güç büyük sorumluluk getirir). Eğer yanlış bir adrese değer yazarsanız C dili sizi durdurmaz (Memory Segfault / Donanım Hard Fault yaşanır!)

× Wild Pointer — Başlatılmamış İşaretçi:

```
int *ptr; // HATA! Adres tanımlanmadı! İçinde "Çöp Veri" bellek adresi var.
*ptr = 10; // TAMAMEN RASTGELE bir yere (Belki Sistem Kernel, Belki Robot Motor pinine!)
// Sistem kesinlikle çökecektir.
```

✅ Çözüm: Bir pointerın şu an bir işi yoksa, ONU ZİNCİRLEYİN (`NULL`).

```
int *ptr = NULL; // NULL adresi 0'dır diyebiliriz, yane "Bir Yeri Gösterme" boş geç komut
if (ptr != NULL) {
    *ptr = 10; // Güvenli (Sadece geçerliyse ateş et)
}
```

5.2 Derleme Hatası Kataloğu (Compilation Errors)

Laboratuvar ortamında en sık karşılaşılan pointer kaynaklı derleme hataları:

#	Hata Mesajı (GCC)	Sebep	Çözüm
1	`makes pointer from integer without a cast`	`&` koymayı unutmak	`fonksiyon(&degisken)` olarak düzeltin
2	`incompatible pointer type`	Tip uyumsuzluğu (`int*` ↔ `float*`)	Pointer ve değişken tiplerini eşleştirin
3	`format '%d' expects 'int' but has type 'int *'`	`printf`'te `%p` yerine `%d` kullanmak	Adres basarken `%p` kullanın
4	`assignment of read-only location`	`const` pointer üzerinden yazma denemesi	`const` kaldırın ya da sadece okuyun

5.3 Mantık Hatası Kataloğu (Logic Errors)

Derleyici hata vermez ancak program beklendiği gibi çalışmaz:

#	Hata	Belirtisi	Çözüm
1	Wild pointer (başlatılmamış)	Program çöker (Segfault)	`int *ptr = NULL;` ile başlatın
2	`&` koymayı unutmak	Fonksiyon orijinali değiştirmez	Adres gönderin: `fonk(&x)`
3	`*` koymayı unutmak (fonk. içinde)	Adres yazdırılır, veri yerine	`*ptr` ile dereference edin
4	Yanlış tipe pointer atamak	Beklenmeyen değerler okunur	Tip uyumunu kontrol edin

```c
// ❌ Yaygın hata: & koymayı unutmak
void guncelle(int *ptr) { *ptr = 50; }

int main() {
    int x = 10;
    // guncelle(x);    // ❌ Kopya gönderildi — hata veya tanımsız davranış
    guncelle(&x);      // ✅ Adres gönderildi — doğru kullanım
    return 0;
}
```

BÖLÜM 6

Uygulamalı Laboratuvar Çalışmaları

6.1 Uygulama 1: İki Sensör Değerinin Takası (swap)

Derste öğrenilen `swap()` fonksiyonunu pekiştirmek amacıyla kullanıcıdan iki değer alan program:

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int gecici = *a;
    *a = *b;
    *b = gecici;
}

int main() {
    int sensor1, sensor2;
    printf("1. sensor degerini girin: ");
    scanf("%d", &sensor1);
    printf("2. sensor degerini girin: ");
    scanf("%d", &sensor2);

    printf("Takas oncesi: S1=%d, S2=%d\n", sensor1, sensor2);
    swap(&sensor1, &sensor2);
```

Pekiştirilen Kavramlar: ✅ Call by Reference, `&` ve `*` kullanımı, `swap` deseni.

6.2 Uygulama 2: Dizinin Min/Max Değerlerini Pointer ile Bulmak

Fonksiyona dizi ve iki pointer gönderilerek, aynı anda minimum ve maksimum değerlerin hesaplanması:

```c
#include <stdio.h>

void minMaxBul(const int *dizi, int boyut, int *min, int *max) {
    *min = dizi[0];
    *max = dizi[0];
    for (int i = 1; i < boyut; i++) {
        if (dizi[i] < *min) *min = dizi[i];
        if (dizi[i] > *max) *max = dizi[i];
    }
}

int main() {
    int sicaklik[6] = {23, 45, 12, 67, 34, 8};
    int enDusuk, enYuksek;

    minMaxBul(sicaklik, 6, &enDusuk, &enYuksek);

```

Pekiştirilen Kavramlar: ✅ Array Decay, `const` pointer, çoklu değer döndürme.

6.3 Uygulama 3: ADC Verisi Normalizasyonu

Ham ADC değerlerini `const` pointer ile okuyup, normalize edilmiş voltajları hesaplayan program:

```c
#include <stdio.h>

void adcNormalize(const int *ham, float *volt, int boyut) {
    for (int i = 0; i < boyut; i++) {
        volt[i] = (ham[i] * 5.0) / 1023.0;  // 10-bit ADC → Volt
    }
}

int main() {
    int adc_ham[5] = {0, 256, 512, 768, 1023};
    float voltaj[5];

    adcNormalize(adc_ham, voltaj, 5);

    for (int i = 0; i < 5; i++) {
        printf("ADC[%d] = %4d  ->  %.3f V\n", i, adc_ham[i], voltaj[i]);
    }
```

Beklenen Çıktı:

```
ADC[0] =    0  ->  0.000 V
```

Özet ve Sonraki Ders

Bu Hafta Öğrendiklerimiz (Mekatroniğin Kalbi):

- **Adres (`&`):** Her değişkenin RAM'de eşsiz bir adresi vardır.
- **Pointer (`*`):** Veri değil, adres tutan özel değişkendir.
- **Call by Reference:** Adres göndererek orijinal veriyi fonksiyon içinden değiştirmek.
- **`swap()` Deseni:** İki değişkenin pointer ile takası — sınavların klasiği.
- **Array Decay:** Dizi isminin aslında 0. elemanın adresi olan gizli bir pointer olması.
- **İşaretçi Aritmetiği:** `ptr + i` ile bellekte adım atmak.
- **`sizeof` ve Pointer Boyutu:** Tüm pointer'lar aynı boyuttadır (platform bağımlı).
- **`const` Pointer:** Veriyi salt okunur olarak koruma.
- **Wild/Null Pointer:** Başlatılmamış pointer tehlikesi ve `NULL` ile güvenli kullanım.

Sonraki Ders — Ön Bakış

Açık Kalan Soru:

> Bir programın çalışma zamanında kaç sensör verisi geleceğini **önceden bilmiyorsak**, sabit boyutlu diziler yeterli olur mu? 1000 sensör bağlandığında `int dizi[1000]` yazmak mı gerekecek?

👉 Çözüm Gelecek Hafta: Dinamik Bellek Yönetimi — `malloc()`, `calloc()`, `realloc()` ve `free()` ile çalışma zamanında bellek ayırma ve geri verme.

Teşekkürler...

Dr. Öğr. Üyesi Hüseyin YÜCE

Marmara Üniversitesi

Mekatronik Mühendisliği Bölümü